

WIPRO SELENIUM MODULE 2

LAB WORKBOOK

Python Unit Testing, PyTest and Selenium Automation

Student Name: Debasmita Majhi

Module: Module 2->Unit Test Frameworks

Module 2 – Lab Workbook

This workbook documents the practical experiments performed in Module 2. Each experiment contains the problem statement, objective, tools, concepts, source code, execution procedure, output placeholder, result and conclusion.

Overall Learning Objectives

- Understand Python unittest and basic assertions.
- Use setUp and tearDown for test preparation and cleanup.
- Create and execute test suites.
- Understand pytest test discovery and execution.
- Use pytest fixtures and selected test execution.
- Integrate Selenium WebDriver with pytest.
- Generate HTML execution reports.
- Understand the Page Object Model for maintainable Selenium automation.

Experiment 2.1 – Unittest – First Test Case

1. Problem Statement

Create a basic Python unittest test case to understand the unittest framework and execute a simple test successfully.

2. Objective

To introduce Python unittest and create the first automated test case.

3. Tools / Software

Python, unittest, VS Code, Command Prompt/PowerShell

4. Concepts Used

unittest framework, TestCase, test methods, assertions

5. Source Code

```
import unittest
```

```
class TestCalculator(unittest.TestCase):
```

```
    def test_addition(self):
```

```
        self.assertEqual(10 + 20, 30)
```

```
if __name__ == "__main__":
```

```
    unittest.main()
```

6. Execution Procedure

Save as experiment_2_1.py and run: python experiment_2_1.py

7. Output

[INSERT TERMINAL OUTPUT SCREENSHOT HERE]

Paste the actual screenshot/output here after running the experiment.

8. Result

Test case executed successfully.

9. Conclusion

Basic unittest automation was implemented and validated.

Experiment 2.2 – setUp, tearDown and Assertions

1. Problem Statement

Create a unittest test case using setUp and tearDown methods and different assertions.

2. Objective

To understand test initialization, cleanup and validation using assertions.

3. Tools / Software

Python, unittest, VS Code

4. Concepts Used

setUp(), tearDown(), assertEquals(), assertTrue(), assertFalse()

5. Source Code

```
import unittest

class TestCalculator(unittest.TestCase):

    def setUp(self):
        print("Test started")

    def tearDown(self):
        print("Test finished")

    def test_addition(self):
        result = 10 + 20
        self.assertEqual(result, 30)

if __name__ == "__main__":
    unittest.main()
```

6. Execution Procedure

Save as experiment_2_2.py and run: python experiment_2_2.py

7. Output

[INSERT TERMINAL OUTPUT SCREENSHOT HERE]

Paste the actual screenshot/output here after running the experiment.

8. Result

All assertion tests passed.

9. Conclusion

setUp and tearDown provide a clear structure for preparation and cleanup.

Experiment 2.3 – Test Suite

1. Problem Statement

Group multiple unittest test cases into a test suite and execute them together.

2. Objective

To understand how multiple test cases can be organized and executed as a suite.

3. Tools / Software

Python, unittest

4. Concepts Used

TestSuite, TestLoader, unittest.TextTestRunner

5. Source Code

```
import unittest

class TestCalculator(unittest.TestCase):

    def test_addition(self):
        self.assertEqual(10 + 20, 30)

    def test_subtraction(self):
        self.assertEqual(30 - 10, 20)

suite = unittest.TestSuite()
suite.addTest(TestCalculator("test_addition"))
suite.addTest(TestCalculator("test_subtraction"))

runner = unittest.TextTestRunner()
runner.run(suite)
```

6. Execution Procedure

Save as experiment_2_3.py and run: python experiment_2_3.py

7. Output

[INSERT TERMINAL OUTPUT SCREENSHOT HERE]

Paste the actual screenshot/output here after running the experiment.

8. Result

Test suite executed successfully.

9. Conclusion

Multiple test cases were grouped and executed as one suite.

Experiment 2.4 – PyTest Introduction and Test Naming

1. Problem Statement

Create basic PyTest test functions using the naming conventions required by PyTest.

2. Objective

To introduce PyTest and demonstrate automatic test discovery.

3. Tools / Software

Python, pytest, VS Code

4. Concepts Used

PyTest, test functions, test discovery

5. Source Code

```
def test_addition():  
    assert 10 + 20 == 30  
  
def test_subtraction():  
    assert 30 - 10 == 20
```

6. Execution Procedure

Save as test_experiment_2_4.py and run: pytest -v

7. Output

[INSERT TERMINAL OUTPUT SCREENSHOT HERE]

Paste the actual screenshot/output here after running the experiment.

8. Result

PyTest discovered and executed the test functions successfully.

9. Conclusion

PyTest provides simple syntax and automatic test discovery.

Experiment 2.5 – PyTest Fixtures

1. Problem Statement

Create and use a PyTest fixture to provide reusable setup data for tests.

2. Objective

To understand reusable test setup using PyTest fixtures.

3. Tools / Software

Python, pytest

4. Concepts Used

pytest.fixture, reusable setup, fixture data

5. Source Code

6. Execution Procedure

Save as test_experiment_2_5.py and run: pytest -v

7. Output

[INSERT TERMINAL OUTPUT SCREENSHOT HERE]

Paste the actual screenshot/output here after running the experiment.

8. Result

```
import pytest
```

```
@pytest.fixture
```

```
def number():
```

```
    return 10
```

```
def test_addition(number):
```

```
    assert number + 20 == 30
```

```
def test_multiplication(number):
```

```
    assert number * 2 == 20
```

9. Conclusion

PyTest fixtures make common test setup reusable and maintainable.

Experiment 2.6 – Multiple Tests and Test Subsets

1. Problem Statement

Create multiple PyTest tests and demonstrate running a selected test subset.

2. Objective

To understand how individual tests or groups of tests can be executed.

3. Tools / Software

Python, pytest

4. Concepts Used

multiple test cases, test selection, pytest command-line execution

5. Source Code

```
def test_addition():  
  
    assert 10 + 20 == 30  
  
  
def test_subtraction():  
  
    assert 30 - 10 == 20  
  
  
def test_multiplication():  
  
    assert 5 * 4 == 20  
  
  
def test_division():  
  
    assert 20 / 5 == 4
```

6. Execution Procedure

Save as test_experiment_2_6.py. Run all tests with: `pytest -v`. Run a selected test with: `pytest -v test_experiment_2_6.py::test_search`

7. Output

[INSERT TERMINAL OUTPUT SCREENSHOT HERE]

Paste the actual screenshot/output here after running the experiment.

8. Result

Multiple tests were executed and a selected test could be run independently.

9. Conclusion

PyTest supports flexible execution of individual tests as well as complete test files.

Experiment 2.7 – Selenium with PyTest

1. Problem Statement

Create a browser automation test using Selenium WebDriver and execute it with PyTest.

2. Objective

To integrate Selenium WebDriver with PyTest for browser automation.

3. Tools / Software

Python, Selenium WebDriver, PyTest, Google Chrome

4. Concepts Used

WebDriver, browser automation, PyTest test function, assertions

5. Source Code

```
from selenium import webdriver
```

```
def test_google_title():
```

```
    driver = webdriver.Chrome()
```

```
    driver.get("https://www.google.com")
```

```
    assert "Google" in driver.title
```

```
    driver.quit()
```

6. Execution Procedure

Save as test_experiment_2_7.py and run: pytest -v test_experiment_2_7.py

7. Output

[INSERT BROWSER/TERMINAL OUTPUT SCREENSHOT HERE]

Paste the actual screenshot/output here after running the experiment.

8. Result

Selenium browser automation test passed.

9. Conclusion

Selenium and PyTest can be combined to create automated browser tests.

Experiment 2.8 – HTML Report

1. Problem Statement

Execute PyTest tests and generate an HTML execution report.

2. Objective

To generate a readable HTML report from PyTest execution results.

3. Tools / Software

Python, pytest, pytest-html

4. Concepts Used

HTML reporting, PyTest report generation

5. Source Code

```
from selenium import webdriver
```

```
def test_google_title():
```

```
    driver = webdriver.Chrome()
```

```
    driver.get("https://www.google.com")
```

```
    assert "Google" in driver.title
```

```
    driver.quit()
```

6. Execution Procedure

Install once: pip install pytest-html. Save as test_experiment_2_8.py. Run: pytest -v --html=report.html

7. Output

[INSERT TERMINAL OUTPUT / HTML REPORT SCREENSHOT HERE]

Paste the actual screenshot/output here after running the experiment.

8. Result

Tests executed successfully and an HTML report was generated.

9. Conclusion

HTML reports provide a convenient record of automated test execution.

Experiment 2.9 – Page Object Model

1. Problem Statement

Create a simple Selenium Page Object Model structure and use a page class from a PyTest test.

2. Objective

To understand how Page Object Model separates page locators/actions from test logic.

3. Tools / Software

Python, Selenium WebDriver, PyTest, Google Chrome

4. Concepts Used

POM, page class, locators, reusable methods

5. Source Code

```
from selenium import webdriver
```

```
class BasePage:
```

```
    def __init__(self, driver):
```

```
        self.driver = driver
```

```
class HomePage(BasePage):
```

```
    def open(self):
```

```
        self.driver.get("https://www.google.com")
```

```
    def get_title(self):
```

```
        return self.driver.title
```

```
def test_home_page():
```

```
    driver = webdriver.Chrome()
```

```
page = HomePage(driver)
```

```
page.open()
```

```
assert "Google" in page.get_title()
```

```
driver.quit()
```

6. Execution Procedure

Save as test_experiment_2_9.py and run: `pytest -v test_experiment_2_9.py`

7. Output

[INSERT BROWSER/TERMINAL OUTPUT SCREENSHOT HERE]

Paste the actual screenshot/output here after running the experiment.

8. Result

Page Object Model test executed successfully.

9. Conclusion

POM separates page-specific locators and actions from test logic, improving reuse and maintenance.

Overall Conclusion

Module 2 provided practical experience with Python unittest, PyTest, test organization, fixtures, Selenium integration, HTML reporting and Page Object Model. These concepts form the foundation for structured and maintainable automated testing.

Final Submission Checklist

- ☐ All Module 2 experiment source files are saved.
- ☐ All experiments were executed successfully.
- ☐ Terminal/browser output screenshots are inserted.
- ☐ HTML report screenshot/file is included for Experiment 2.8.
- ☐ Experiment results and conclusions are completed.
- ☐ Workbook is reviewed before submission.